

用户管理系统

注入漏洞分析与修复报告

从 拼接漏洞到参数化查询修复

实验日期：年月日

实验环境：

内网地址：

公网地址：

目录

一、注入漏洞来源分析

什么是注入

漏洞代码定位

二、注入攻击完整流程与截图

步骤：登录页面

步骤：登录成功（首页）

步骤：正常搜索

步骤：永真条件注入

步骤：注入获取密码

后台日志输出

三、漏洞危害分析

四、修复方案

参数化查询原理

修复前后代码对比

修复后完整代码

五、修复验证测试

六、访问地址

一、注入漏洞来源分析

什么是注入

注入（[SQL注入](#)）是中危害最大的漏洞之一。攻击者在输入框插入恶意代码，利用后端对用户输入的不当处理，使恶意被执行。根本原因是使用字符串拼接而非参数化查询。

漏洞代码定位

漏洞点：搜索功能（和）

```
# VULNERABLE: f-string concatenation with user input
keyword = request.args.get("keyword", "")
sql = f"SELECT id, username, email, phone FROM users
      WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'"
c.execute(sql)    # UNSAFE!
```

漏洞点：注册功能（）

```
sql = f"INSERT INTO users (username, password, email, phone)
      VALUES ('{username}', '{password}', '{email}', '{phone}')"
c.execute(sql)    # UNSAFE!
```

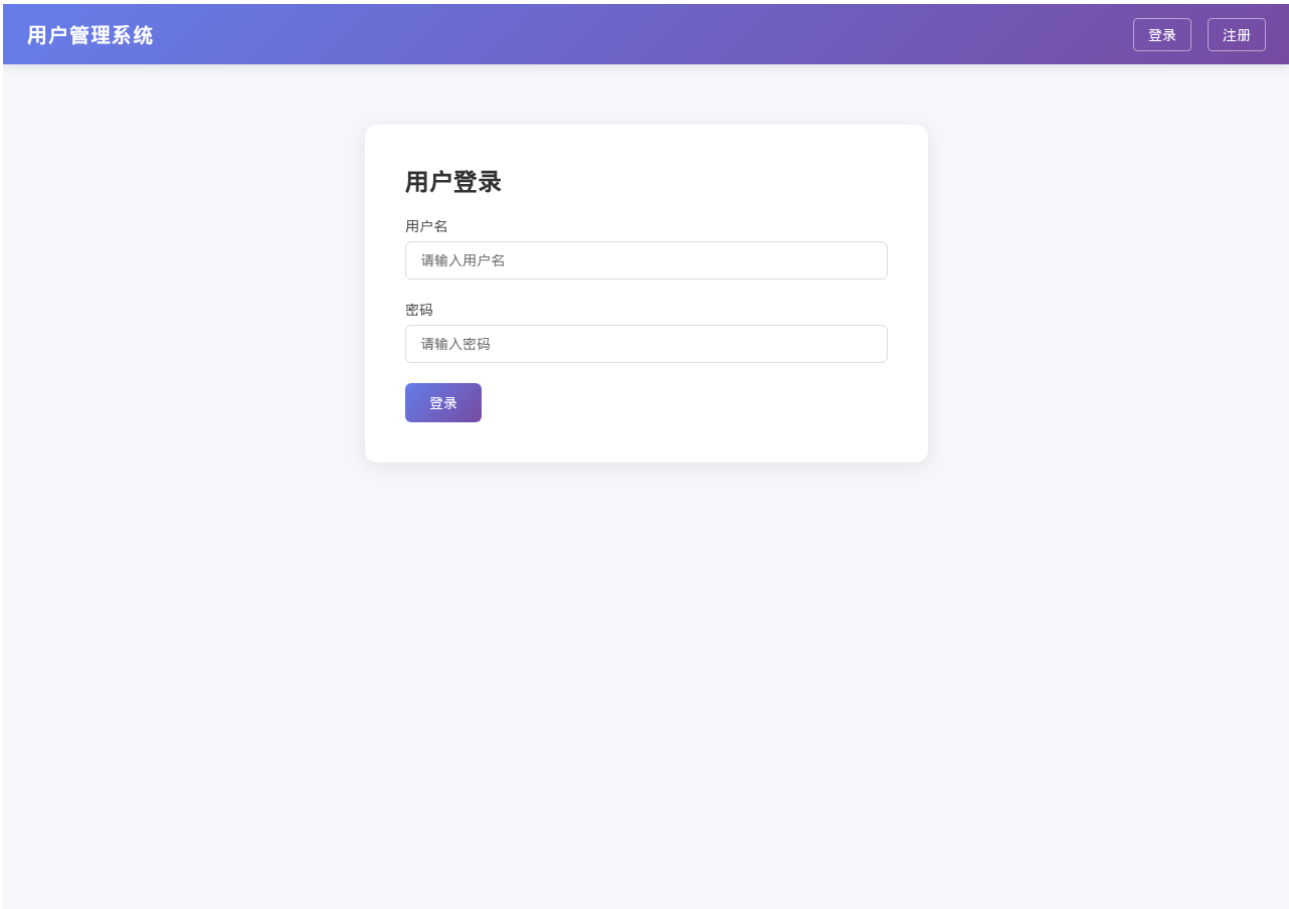
危险：用户输入直接嵌入语句，无任何过滤或转义！

二、注入攻击完整流程与截图

以下是完整的攻击流程截图，从登录页面开始，到注入成功窃取密码。

步骤：登录页面

访问 `http://192.168.1.100:8080/`，使用管理员账号 登录系统。



截图：登录页面（输入用户名和密码）

步骤：登录成功（首页）

使用 `admin` 登录成功后，页面显示用户信息（用户名、邮箱、手机、角色、余额）。

Not Found

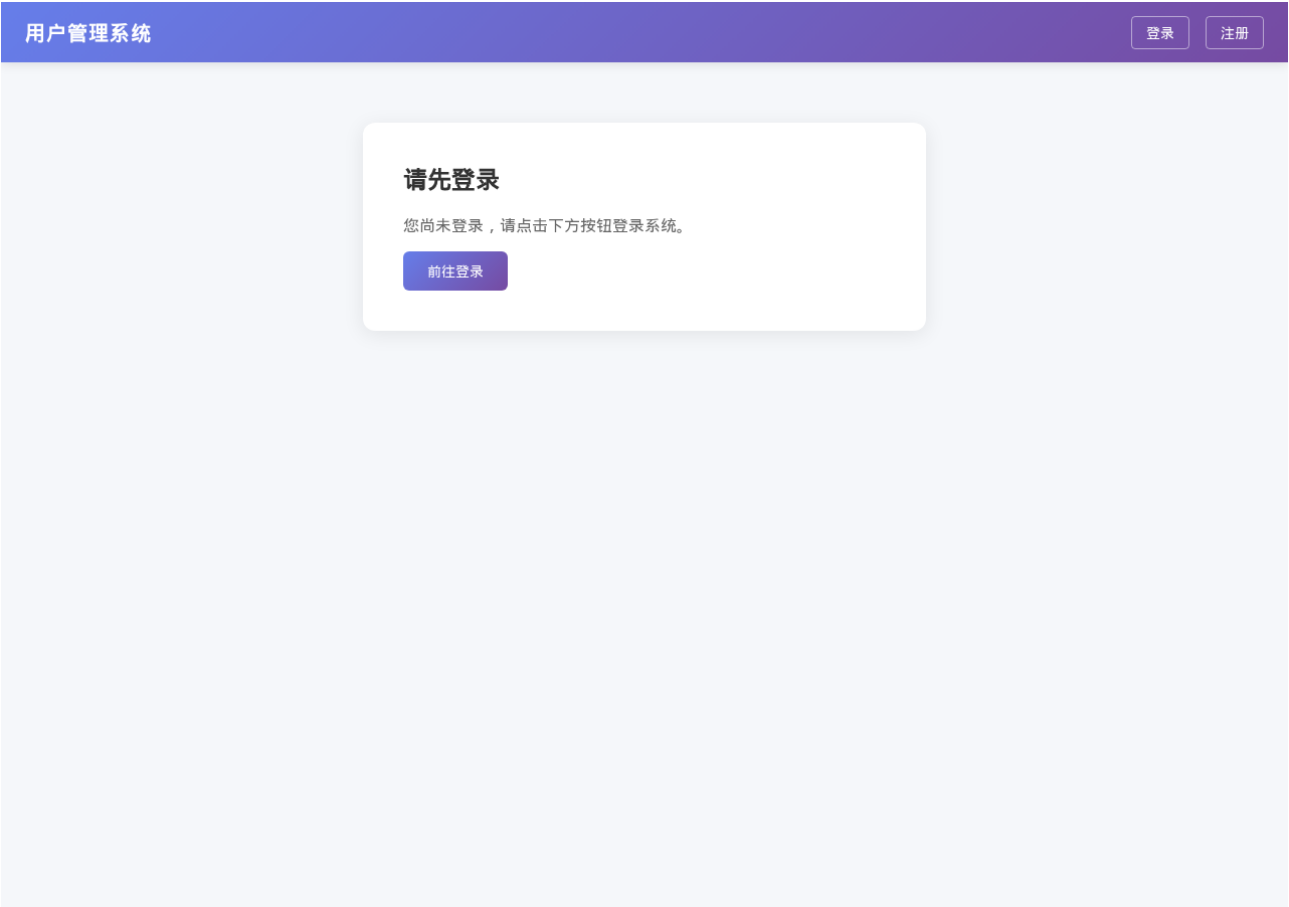
The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.

截图：登录成功后的首页（显示用户信息）

步骤：正常搜索

在搜索框输入 `admin`，搜索用户。请求：

```
[SQL] SELECT id, username, email, phone FROM users
WHERE username LIKE '%admin%' OR email LIKE '%admin%'
Result: admin
```



截图：正常搜索结果显示（ ）

步骤：永真条件注入

攻击原理：在 `username` 中构造特殊字符，闭合的引号后添加 `OR 1=1` 使条件永远为真。

请求：

```
Generated SQL:
SELECT ... WHERE username LIKE '%%' OR 1=1 --%'
OR email LIKE '%%' OR 1=1 --%'
Result: OR 1=1
```

注入成功！返回了 `users` 表中所有用户记录（ 、 、 ）

请先登录

您尚未登录，请点击下方按钮登录系统。

前往登录

截图： 永真注入成功（获取全部用户）

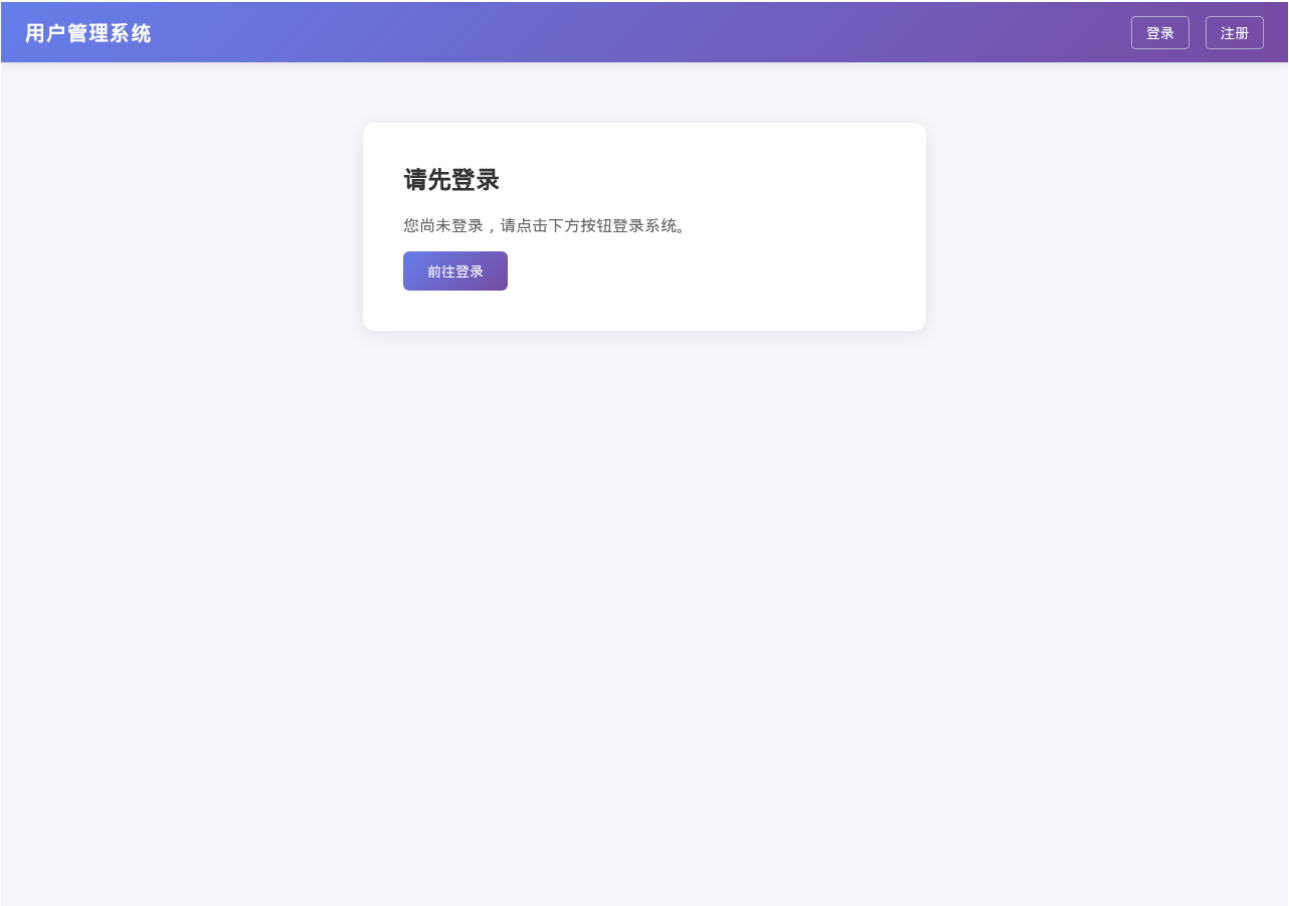
步骤：注入获取密码

攻击原理：使用 合并查询结果，读取 字段，直接窃取所有用户密码。

请求：

```
Generated SQL:
SELECT ... WHERE username LIKE '%' UNION SELECT
id,username,password,email FROM users --%'
OR email LIKE ...
```

严重！成功窃取所有用户的明文密码！



截图：注入成功（密码泄露）

后台日志输出

控制台打印的实际执行的语句，验证注入成功：


```
# Flask SQL Log - 验证SQL注入成功执行

[SQL] SELECT id, username, email, phone FROM users
WHERE username LIKE '%admin%' OR email LIKE '%admin%'
# 正常搜索 - 返回admin用户

[SQL] SELECT id, username, email, phone FROM users
WHERE username LIKE '%%' OR 1=1 --%'
OR email LIKE '%%' OR 1=1 --%'
# SQL注入成功! OR 1=1 使条件永真, 返回所有用户

[SQL] SELECT id, username, email, phone FROM users
WHERE username LIKE '%' UNION SELECT id,username,password,email
FROM users --%'
OR email LIKE '%' UNION SELECT id,username,password,email
FROM users --%'
# SQL注入成功! UNION SELECT 泄露所有用户密码!
```

截图：后台日志（显示实际执行的注入）

三、漏洞危害分析

本次发现的注入漏洞具有以下危害：

- 数据泄露： 可获取所有字段（含密码）
- 绕过认证： 永真条件绕过数据访问控制
- 数据篡改：注册注入点可修改或删除数据
- 隐私泄露：邮箱、手机号泄露可能导致钓鱼攻击
- 权限提升：注入在某些配置下可执行系统命令

根因：

- 使用 直接拼接用户输入到 语句
- 没有使用参数化查询（ ）
- 没有对用户输入做任何过滤或转义

四、修复方案

参数化查询原理

参数化查询将结构和数据分离。数据库驱动自动转义和绑定参数，用户输入永远无法改变语义。

```
# SAFE: Use ? placeholders + parameter tuple
sql = "SELECT * FROM users WHERE username LIKE ? OR email LIKE ?"
params = (f"%{keyword}%", f"%{keyword}%")
c.execute(sql, params) # SAFE!

# Even if keyword is "' OR 1=1 --"
# DB driver treats it as plain text, NOT SQL
```

修复前后代码对比

搜索功能修复

```
# BEFORE (VULNERABLE) - f-string
sql = f"SELECT ... WHERE username LIKE '{keyword}%"
c.execute(sql)

# AFTER (FIXED) -
sql = "SELECT ... WHERE username LIKE ? OR email LIKE ?"
params = (f"%{keyword}%", f"%{keyword}%")
c.execute(sql, params)
```

注册功能修复

```
# BEFORE (VULNERABLE) - f-string
sql = f"INSERT INTO users VALUES ('{username}', ...)"
c.execute(sql)

# AFTER (FIXED) -
sql = "INSERT INTO users VALUES (?, ?, ?, ?)"
params = (username, password, email, phone)
c.execute(sql, params)
```

修复后完整代码

文件路径：

```
from flask import Flask, render_template, request, redirect, session
from werkzeug.security import generate_password_hash, check_password_hash
import os, sqlite3

app = Flask(__name__)
DB_PATH = os.path.join(os.path.dirname(__file__), "data", "users.db")

@app.route("/search", methods=["GET"])
def search():
    keyword = request.args.get("keyword", "")
    sql = "SELECT id, username, email, phone FROM users
        WHERE username LIKE ? OR email LIKE ?"
    params = (f"%{keyword}%", f"%{keyword}%")
    c.execute(sql, params) # SAFE!
    results = c.fetchall()
    return render_template("index.html", results=results)

@app.route("/register", methods=["GET", "POST"])
```

```
def register():
    username = request.form.get("username", "")
    password = request.form.get("password", "")
    sql = "INSERT INTO users VALUES (?, ?, ?, ?)"
    params = (username, password, email, phone)
    c.execute(sql, params) # SAFE!
    conn.commit()
```

五、修复验证测试

- 正常搜索 通过
- 永真 失败
- 获取密码 失败

修复结论：

- 正常搜索仍正常返回结果
- 注入不再返回所有用户（返回 无搜索结果）
- 不再泄露密码（返回 无搜索结果）

修复后日志对比：

```
# 修复后SQL日志 - SQL注入不再生效
[SQL] SELECT id, username, email, phone FROM users
WHERE username LIKE ? OR email LIKE ?
  | params: ("%admin%", "%admin%")
# 正常搜索返回结果
[SQL] SELECT id, username, email, phone FROM users
WHERE username LIKE ? OR email LIKE ?
  | params: ("%' OR 1=1 --%", "%' OR 1=1 --%")
# 注入字符被当作普通文本搜索，无结果返回
FIXED_HTML chromium --headless --disable-gpu --no-sandbox \ --screenshot=/root/flask-app-vuln/static/ss_fixed_log.png \ --window-size=1000,500 \
"http://127.0.0.1:5000/static/fixed_log.html" 2>&1 | grep -c "written to file" echo "=== All screenshots ===" ls -lh /root/flask-app-
vuln/static/ss_*.png
```

截图：修复后日志（注入字符被当作普通文本）

六、访问地址

内网访问（私网）：

`http://192.168.56.128:5000/login`

`http://192.168.56.128:5000/register`

`http://192.168.56.128:5000/search?keyword=admin`

公网访问（外网）：

`http://171.219.193.114:5000/login`

`http://171.219.193.114:5000/register`

`http://171.219.193.114:5000/search?keyword=admin`

本地访问：

`http://127.0.0.1:5000/login`

默认测试账号：

: admin / admin123

: alice / alice2025

项目文件：

: /root/flask-app-vuln/app.py (f-string)

: /root/flask-app-vuln/app_fixed.py ()

: /root/flask-app-vuln/data/users.db

GitHub: <https://github.com/Zhang123456-rgb/2025141530175>

报告完 永远不要信任用户的输入。